

EN.553.761 Unconstrained Nonlinear Optimization

Jonathan Y. Ma

June 2026

This course was taught by Dr. Shiqian Ma in the Fall 2026 Semester. The notes are transcribed by Jonathan Ma and may contain errors and omitted content, so any issues are to be directed towards me.

Contents

0.1	Gradient Method	1
0.2	Quasi-Newton Method	2
0.3	Line Search	3
0.4	Subgradient Method	4
0.5	Proximal Gradient Method	4
0.6	Nesterov's Accelerated Gradient Method	5
0.7	Frank-Wolfe Method	5
0.8	Proximal Point Algorithm	6
0.9	Augmented Lagrangian Method	6
0.10	Alternating Direction Method of Multipliers	7
0.11	Coordinate Descent Method	7
0.12	Stochastic Gradient Descent	8
0.13	Adaptive Methods	8
0.13.1	ADAM	9
0.13.2	AdaGrad	9
0.14	Shampoo and Muon	9
0.15	Automatic Differentiation	10
0.16	Applications	11
0.16.1	Matrix Factorization	11
0.16.2	Support Vector Machines	11
0.16.3	Compressed Sensing	12
0.16.4	Deep Learning	12

0.1 Gradient Method

Consider

$$\min_{x \in \mathbb{R}^d} f(x),$$

where f is differentiable. A point x^* is a global minimizer if $f(x^*) \leq f(x)$ for every x , a local minimizer if this holds in a neighborhood, and a stationary point if $\nabla f(x^*) = 0$.

Definition 0.1.1 (Smoothness and strong convexity). The function f is L -smooth if

$$\|\nabla f(x) - \nabla f(y)\| \leq L \|x - y\|.$$

It is μ -strongly convex if

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \frac{\mu}{2} \|y - x\|^2.$$

For twice differentiable f , the sufficient Hessian conditions are $\nabla^2 f(x) \preceq LI$ for smoothness and $\nabla^2 f(x) \succeq \mu I$ for strong convexity. Convexity makes every local minimum global; strong convexity makes the minimizer unique.

Lemma 0.1.1 (Descent lemma). *If f is L -smooth, then*

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|^2.$$

Proof. Apply the fundamental theorem of calculus to $f(x + t(y - x))$ and bound the change in the gradient using Lipschitz continuity. $\square$

Gradient descent uses

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k). \quad (0.1.1)$$

With $\alpha = 1/L$, the descent lemma gives

$$f(x_{k+1}) \leq f(x_k) - \frac{1}{2L} \|\nabla f(x_k)\|^2.$$

Theorem 0.1.1 (Basic gradient-descent rates). *Let f be convex and L -smooth, and let x^* be a minimizer. Gradient descent with $\alpha = 1/L$ satisfies*

$$f(x_k) - f(x^*) \leq \frac{L \|x_0 - x^*\|^2}{2k}.$$

If f is also μ -strongly convex, then

$$f(x_k) - f(x^*) \leq \left(1 - \frac{\mu}{L}\right)^k \{f(x_0) - f(x^*)\}.$$

The condition number $\kappa = L/\mu$ controls the linear rate. Poor feature scaling produces elongated level sets and slow zig-zagging. Stop using a combination of gradient norm, relative objective change, step norm, and an iteration budget; objective change alone can be misleading on flat regions.

0.2 Quasi-Newton Method

Newton's method locally approximates

$$f(x + p) \approx f(x) + g^\top p + \frac{1}{2} p^\top H p$$

and solves $Hp = -g$, where $g = \nabla f(x)$ and $H = \nabla^2 f(x)$. Near a nondegenerate minimizer, full Newton steps converge quadratically under standard Lipschitz-Hessian assumptions. Far from a minimizer, H can be indefinite and the Newton direction need not descend.

Quasi-Newton methods replace the Hessian or inverse Hessian with an estimate. Set $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$. The secant condition is

$$B_{k+1}s_k = y_k.$$

The BFGS update is

$$B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k}.$$

If $B_k \succ 0$ and $y_k^\top s_k > 0$, then $B_{k+1} \succ 0$. Wolfe line search guarantees this curvature condition for a descent direction.

In inverse form, with $\rho_k = (y_k^\top s_k)^{-1}$,

$$H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top,$$

and the direction is $p_k = -H_k g_k$. L-BFGS stores only the last few (s_k, y_k) pairs and applies the inverse implicitly, making memory $O(md)$ instead of $O(d^2)$.

Theorem 0.2.1 (Local BFGS behavior). *For a twice continuously differentiable, strongly convex objective with Lipschitz Hessian near x^* , BFGS with a suitable Wolfe line search converges superlinearly under the standard Dennis–Moré conditions.*

In computation, solve Newton systems rather than invert Hessians. Regularize an indefinite Hessian using $H + \lambda I$, use a trust region, or switch to a descent direction.

0.3 Line Search

Given a descent direction p_k with $g_k^\top p_k < 0$, line search chooses $\alpha_k > 0$ and sets $x_{k+1} = x_k + \alpha_k p_k$. Exact line search minimizes $\phi(\alpha) = f(x_k + \alpha p_k)$ but is usually unnecessary.

Definition 0.3.1 (Armijo and Wolfe conditions). The Armijo sufficient-decrease condition is

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha g_k^\top p_k, \quad 0 < c_1 < 1.$$

The curvature condition is

$$\nabla f(x_k + \alpha p_k)^\top p_k \geq c_2 g_k^\top p_k, \quad c_1 < c_2 < 1.$$

The strong Wolfe condition replaces the latter with an absolute-value bound.

Backtracking starts from an initial step, often one, and repeatedly multiplies it by $\rho \in (0, 1)$ until Armijo holds.

Algorithm 1 Backtracking line search

```

1: Choose  $\alpha \leftarrow \alpha_0$ ,  $\rho \in (0, 1)$ ,  $c_1 \in (0, 1)$ .
2: while  $f(x + \alpha p) > f(x) + c_1 \alpha \nabla f(x)^\top p$  do
3:    $\alpha \leftarrow \rho \alpha$ .
4: end while
5: Return  $\alpha$ .

```

Proposition 0.3.1. *If ∇f is L -Lipschitz and p is a descent direction, backtracking terminates after finitely many reductions.*

The Wolfe conditions prevent steps that are too short and support global convergence theory for nonlinear conjugate-gradient and quasi-Newton methods.

0.4 Subgradient Method

For a convex, possibly nondifferentiable function f , a vector g is a subgradient at x if

$$f(y) \geq f(x) + g^\top(y - x) \quad \text{for all } y.$$

The set $\partial f(x)$ of all subgradients is the subdifferential. For $f(x) = |x|$, $\partial f(0) = [-1, 1]$. A point minimizes a proper convex function exactly when $0 \in \partial f(x)$.

The subgradient iteration is

$$x_{k+1} = x_k - \alpha_k g_k, \quad g_k \in \partial f(x_k).$$

Unlike a gradient, $-g_k$ need not be a descent direction, so objective values may oscillate.

Theorem 0.4.1 (Subgradient bound). *Suppose $\|g_k\| \leq G$ and $\|x_0 - x^*\| \leq R$. Then*

$$\min_{0 \leq j < k} \{f(x_j) - f(x^*)\} \leq \frac{R^2 + G^2 \sum_{j=0}^{k-1} \alpha_j^2}{2 \sum_{j=0}^{k-1} \alpha_j}.$$

Thus steps satisfying $\sum_k \alpha_k = \infty$ and $\sum_k \alpha_k^2 < \infty$ give convergence of the best objective value.

With a horizon known in advance, constant steps of order $1/\sqrt{k}$ give the characteristic $O(k^{-1/2})$ rate. Proximal methods are usually much faster when the nonsmooth term has a tractable proximal map.

0.5 Proximal Gradient Method

Consider the composite convex problem

$$\min_x F(x) = f(x) + g(x),$$

where f is differentiable and L -smooth while g is convex and may be nonsmooth.

Definition 0.5.1 (Proximal operator). For $\alpha > 0$,

$$\text{prox}_{\alpha g}(v) = \arg \min_x \left\{ g(x) + \frac{1}{2\alpha} \|x - v\|^2 \right\}.$$

The proximal-gradient step is

$$x_{k+1} = \text{prox}_{\alpha g}\{x_k - \alpha \nabla f(x_k)\}. \quad (0.5.1)$$

It minimizes a quadratic upper model of f plus the exact g . Examples are

$$\text{prox}_{\alpha \lambda \|\cdot\|_1}(v) = \text{sign}(v)(|v| - \alpha \lambda)_+, \quad \text{prox}_{\mathbb{1}_C}(v) = \Pi_C(v).$$

Theorem 0.5.1 (Proximal-gradient rate). *If F is convex, f is L -smooth, and a minimizer exists, then with $\alpha = 1/L$,*

$$F(x_k) - F(x^*) \leq \frac{L \|x_0 - x^*\|^2}{2k}.$$

The gradient mapping $G_\alpha(x) = \alpha^{-1}[x - \text{prox}_{\alpha g}(x - \alpha \nabla f(x))]$ generalizes the gradient: $G_\alpha(x) = 0$ exactly at a first-order solution.

0.6 Nesterov's Accelerated Gradient Method

Acceleration combines a gradient step with momentum. For convex L -smooth f , one common form is

$$x_{k+1} = y_k - \frac{1}{L} \nabla f(y_k), \quad y_{k+1} = x_{k+1} + \frac{k}{k+3}(x_{k+1} - x_k).$$

Theorem 0.6.1 (Accelerated rate). *Nesterov's method can be chosen so that*

$$f(x_k) - f(x^*) \leq \frac{2L \|x_0 - x^*\|^2}{(k+1)^2}.$$

For μ -strongly convex f , accelerated methods achieve a linear factor of order $1 - 1/\sqrt{\kappa}$ rather than $1 - 1/\kappa$.

FISTA applies the same idea to proximal gradient and achieves $O(k^{-2})$ for composite convex objectives. Momentum can cause nonmonotone objectives and oscillation. Adaptive restart resets momentum when the objective or gradient geometry indicates overshooting.

0.7 Frank-Wolfe Method

Frank-Wolfe solves $\min_{x \in C} f(x)$ for a compact convex set C using a linear minimization oracle rather than projection:

$$s_k \in \arg \min_{s \in C} \nabla f(x_k)^\top s, \quad x_{k+1} = (1 - \gamma_k)x_k + \gamma_k s_k.$$

It is useful when linear optimization over C is cheap but Euclidean projection is expensive, as for nuclear-norm balls.

The Frank–Wolfe gap

$$g_{\text{FW}}(x) = \max_{s \in C} \nabla f(x)^\top (x - s)$$

is computable and satisfies $f(x) - f(x^*) \leq g_{\text{FW}}(x)$ for convex f . It is both a stopping criterion and a certificate.

Theorem 0.7.1 (Frank–Wolfe rate). *If f is convex and L -smooth on compact C with diameter D , then with $\gamma_k = 2/(k+2)$,*

$$f(x_k) - f(x^*) \leq \frac{2LD^2}{k+2}.$$

Iterates are convex combinations of oracle outputs, often giving sparse or low-rank solutions early. Standard Frank–Wolfe can zig-zag near a boundary; away-step variants achieve faster rates under additional geometry.

0.8 Proximal Point Algorithm

The proximal point algorithm for a proper closed convex f is

$$x_{k+1} = \text{prox}_{\lambda_k f}(x_k) = \arg \min_x \left\{ f(x) + \frac{1}{2\lambda_k} \|x - x_k\|^2 \right\}. \quad (0.8.1)$$

Its optimality condition is $0 \in \partial f(x_{k+1}) + \lambda_k^{-1}(x_{k+1} - x_k)$, or $x_k - x_{k+1} \in \lambda_k \partial f(x_{k+1})$.

Theorem 0.8.1 (Fejér monotonicity). *If x^* minimizes f , then every exact proximal point step satisfies*

$$\|x_{k+1} - x^*\|^2 \leq \|x_k - x^*\|^2 - \|x_{k+1} - x_k\|^2.$$

If $\inf_k \lambda_k > 0$ and a minimizer exists, the iterates converge to a minimizer in finite dimensions.

PPA is robust but each step may be as difficult as the original problem. Its importance is conceptual: proximal gradient, augmented Lagrangian, and ADMM can be interpreted as tractable approximations or operator-splitting forms of proximal point iterations.

0.9 Augmented Lagrangian Method

For equality constraints

$$\min_x f(x) \quad \text{subject to} \quad Ax = b,$$

the augmented Lagrangian is

$$\mathcal{L}_\rho(x, y) = f(x) + y^\top (Ax - b) + \frac{\rho}{2} \|Ax - b\|^2. \quad (0.9.1)$$

The method of multipliers alternates

$$x_{k+1} \in \arg \min_x \mathcal{L}_\rho(x, y_k), \quad y_{k+1} = y_k + \rho(Ax_{k+1} - b).$$

The quadratic penalty improves feasibility without requiring $\rho \rightarrow \infty$, while the dual variable accumulates constraint violations.

Under convexity and standard constraint qualifications, a pair (x^*, y^*) is optimal when

$$0 \in \partial f(x^*) + A^\top y^*, \quad Ax^* = b.$$

These KKT residuals should be monitored. Inner minimization need not be exact, but its tolerances must become sufficiently accurate for convergence theory.

0.10 Alternating Direction Method of Multipliers

ADMM addresses

$$\min_{x,z} f(x) + g(z) \quad \text{subject to} \quad Ax + Bz = c.$$

Using the scaled dual variable $u = y/\rho$, the iterations are

$$\begin{aligned} x_{k+1} &= \arg \min_x \left\{ f(x) + \frac{\rho}{2} \|Ax + Bz_k - c + u_k\|^2 \right\}, \\ z_{k+1} &= \arg \min_z \left\{ g(z) + \frac{\rho}{2} \|Ax_{k+1} + Bz - c + u_k\|^2 \right\}, \\ u_{k+1} &= u_k + Ax_{k+1} + Bz_{k+1} - c. \end{aligned}$$

Separating x and z can turn a difficult joint problem into two simple subproblems.

Definition 0.10.1 (ADMM residuals). The primal residual is $r_{k+1} = Ax_{k+1} + Bz_{k+1} - c$. In the common constraint $x = z$, the dual residual is $s_{k+1} = \rho(z_{k+1} - z_k)$, up to the appropriate matrix factor in the general form.

Stop only when both residuals are small relative to absolute and relative tolerances. A large ρ emphasizes feasibility but can slow dual progress; a small ρ does the reverse. Residual balancing adapts ρ while rescaling u consistently.

Theorem 0.10.1 (Convex ADMM convergence). *If f and g are proper, closed, convex, a saddle point exists, and the subproblems are solved exactly, then the primal residual converges to zero, objective values converge to the optimum, and dual variables converge under standard rank conditions.*

0.11 Coordinate Descent Method

Coordinate descent updates one block at a time:

$$x_j^{k+1} \in \arg \min_t f(x_1^{k+1}, \dots, x_{j-1}^{k+1}, t, x_{j+1}^k, \dots, x_d^k).$$

Updates may be cyclic, randomized, greedy, or performed in blocks. For a smooth objective with coordinate Lipschitz constant L_j , a coordinate gradient step is $x_j \leftarrow x_j - L_j^{-1} \partial_j f(x)$.

Theorem 0.11.1 (Randomized coordinate rate). *For convex smooth f with coordinate-wise Lipschitz gradients, randomized coordinate descent has an expected $O(1/k)$ objective gap. Under strong convexity it converges linearly in expectation, with a rate controlled by the coordinate smoothness constants.*

Coordinate methods are effective when updates are closed form and data are sparse. For lasso, each update is soft thresholding. Cache residuals rather than recomputing $X\beta$ after each coordinate. Pure coordinate minimization can fail for coupled nonsmooth nonconvex objectives, so assumptions matter.

0.12 Stochastic Gradient Descent

For finite-sum or expectation objectives,

$$f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x) \quad \text{or} \quad f(x) = \mathbb{E}_\xi[F(x; \xi)],$$

stochastic gradient descent (SGD) uses

$$x_{k+1} = x_k - \alpha_k g_k, \quad \mathbb{E}[g_k \mid x_k] = \nabla f(x_k). \quad (0.12.1)$$

A minibatch average lowers gradient variance roughly inversely with batch size until correlations and hardware effects dominate.

Theorem 0.12.1 (Strongly convex SGD rate). *Suppose f is μ -strongly convex, stochastic gradients are unbiased with bounded second moment, and α_k is of order $1/k$. Then*

$$\mathbb{E}[f(\bar{x}_k) - f(x^*)] = O(k^{-1})$$

for a suitable averaged iterate. For convex but not strongly convex objectives, the standard rate is $O(k^{-1/2})$.

A constant learning rate reaches a noise neighborhood rather than converging exactly, but is useful for rapid initial progress. Decay reduces the steady-state variance. Momentum updates

$$v_{k+1} = \beta v_k + g_k, \quad x_{k+1} = x_k - \alpha v_{k+1}$$

smooth noisy directions and accelerate persistent ones.

Shuffle finite datasets between epochs. Report gradient evaluations or data passes, not iterations alone, when comparing batch sizes. For nonconvex smooth objectives, a common result controls the average squared gradient norm, so convergence means approaching stationarity rather than a global minimum.

0.13 Adaptive Methods

Adaptive methods rescale coordinates using past gradients:

$$x_{k+1} = x_k - \alpha D_k^{-1} m_k.$$

They are helpful for sparse gradients and poorly scaled coordinates. However, their implicit geometry, memory cost, and generalization can differ from SGD. The stabilizer ε is part of the algorithm, not merely a guard against division by zero.

0.13.1 ADAM

Adam maintains exponential first and second moments:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1 - \beta_1) g_k, \\ v_k &= \beta_2 v_{k-1} + (1 - \beta_2) g_k^{\odot 2}, \\ \widehat{m}_k &= m_k / (1 - \beta_1^k), \quad \widehat{v}_k = v_k / (1 - \beta_2^k), \\ x_{k+1} &= x_k - \alpha \frac{\widehat{m}_k}{\sqrt{\widehat{v}_k} + \varepsilon}. \end{aligned}$$

Bias correction matters early because moments start at zero.

Vanilla Adam does not converge for every convex online problem; AMSGrad repairs a problematic decrease in the effective second-moment accumulator by using a coordinatewise running maximum. In deep learning, AdamW decouples weight decay from the adaptive gradient update. Adding λx to the gradient is not equivalent to multiplicative weight decay under a coordinate-dependent preconditioner.

0.13.2 AdaGrad

Diagonal AdaGrad uses

$$v_k = v_{k-1} + g_k^{\odot 2}, \quad x_{k+1} = x_k - \alpha \frac{g_k}{\sqrt{v_k} + \varepsilon}.$$

Coordinates with frequent large gradients receive smaller future steps.

Theorem 0.13.1 (AdaGrad regret, informal). *For convex online losses and a bounded domain, diagonal AdaGrad has regret controlled by*

$$\sum_{j=1}^d \sqrt{\sum_{k=1}^T g_{k,j}^2},$$

up to a factor depending on the domain diameter. This can be much smaller than a bound based on the worst gradient when gradients are sparse.

Because v_k only increases, learning rates may decay too aggressively in a long nonstationary run. RMSProp and Adam use exponential rather than cumulative second moments.

0.14 Shampoo and Muon

Diagonal adaptive methods ignore correlation between parameter coordinates. Full-matrix AdaGrad captures it but requires prohibitive storage and matrix roots. Shampoo exploits

tensor structure. For a matrix gradient $G_k \in \mathbb{R}^{m \times n}$, a basic form accumulates

$$L_k = \varepsilon I_m + \sum_{t \leq k} G_t G_t^\top, \quad R_k = \varepsilon I_n + \sum_{t \leq k} G_t^\top G_t$$

and preconditions by

$$\tilde{G}_k = L_k^{-1/4} G_k R_k^{-1/4}. \quad (0.14.1)$$

Higher-order tensors receive one preconditioner per mode. Matrix roots are expensive, so practical implementations update them intermittently, use blocking, and combine them with grafting or momentum. Damping controls nearly singular eigendirections.

Muon is designed primarily for matrix-valued hidden-layer weights. It forms a momentum matrix M_k and approximately replaces its singular values by one,

$$M_k = U \Sigma V^\top \quad \mapsto \quad \text{polar}(M_k) = UV^\top,$$

usually through a small number of Newton–Schulz matrix iterations rather than an SVD. The resulting direction is then scaled and used for the parameter update. Vector parameters and some embedding or output parameters are commonly handled by another optimizer.

Shampoo uses accumulated second-moment geometry; Muon orthogonalizes a current momentum direction. They are therefore not interchangeable despite both using matrix operations. Exact scaling conventions vary by implementation, so a reproducible description must specify normalization, Newton–Schulz polynomial, iteration count, momentum, weight decay, and parameter groups.

0.15 Automatic Differentiation

Automatic differentiation (AD) applies the chain rule to a computational graph. It is neither symbolic differentiation nor finite differences. For $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with Jacobian $J_f(x)$:

- forward mode computes a Jacobian–vector product $J_f(x)v$;
- reverse mode computes a vector–Jacobian product $u^\top J_f(x)$.

Forward mode is attractive when inputs are few; reverse mode is attractive for a scalar loss with many parameters, because one reverse sweep obtains the full gradient.

Example 0.15.1 (Reverse-mode chain rule). If $z = Wx$, $a = \tanh z$, and $\ell = \frac{1}{2} \|a - y\|^2$, reverse mode computes

$$\bar{a} = a - y, \quad \bar{z} = \bar{a} \odot (1 - \tanh^2 z), \quad \nabla_W \ell = \bar{z} x^\top.$$

The barred variables are adjoints: derivatives of the final loss with respect to intermediate values.

Reverse mode stores intermediate activations, trading memory for computation. Checkpointing recomputes selected activations during the backward pass. Higher-order derivatives are

obtained by composing modes; Hessian–vector products can be computed without forming the Hessian.

Common bugs arise from in-place mutation, unintended detachment of the graph, nondifferentiable control flow, and unstable primitives. Gradient checking with centered finite differences is useful on small problems, but step sizes that are too small suffer cancellation and those too large suffer truncation error.

0.16 Applications

Optimization algorithms should be chosen from problem structure: smoothness, convexity, separability, sparsity, constraint geometry, and data-access cost. The following examples connect the general methods to recurring machine learning and inverse-problem formulations.

0.16.1 Matrix Factorization

Given a partially observed matrix M , low-rank factorization solves

$$\min_{U \in \mathbb{R}^{m \times r}, V \in \mathbb{R}^{n \times r}} \frac{1}{2} \sum_{(i,j) \in \Omega} (M_{ij} - u_i^\top v_j)^2 + \frac{\lambda}{2} (\|U\|_F^2 + \|V\|_F^2). \quad (0.16.1)$$

The problem is nonconvex jointly but convex in either factor with the other fixed. Alternating least squares exploits this biconvex structure. SGD samples an observed entry and updates only u_i and v_j :

$$e_{ij} = M_{ij} - u_i^\top v_j, \quad u_i \leftarrow u_i + \alpha(e_{ij}v_j - \lambda u_i), \quad v_j \leftarrow v_j + \alpha(e_{ij}u_i - \lambda v_j),$$

where the second update should use either the old u_i or a documented sequential convention.

The factors are nonidentifiable: $UV^\top = (UA)(VA^{-\top})^\top$ for invertible A . Prediction of the product may be well defined even when individual factors are not. Random initialization, regularization, and validation of rank r are essential.

The convex alternative minimizes a data-fit term plus nuclear norm $\|X\|_* = \sum_j \sigma_j(X)$. Its proximal operator performs singular-value soft thresholding, while Frank–Wolfe on a nuclear-norm ball adds rank-one directions using leading singular vectors.

0.16.2 Support Vector Machines

The linear soft-margin SVM has a smooth quadratic regularizer and nonsmooth hinge losses:

$$\min_{\beta_0, \beta} \frac{\lambda}{2} \|\beta\|^2 + \frac{1}{n} \sum_{i=1}^n [1 - y_i(\beta_0 + x_i^\top \beta)]_+.$$

Subgradient methods are simple but slow. Coordinate descent is effective in the dual; stochastic subgradient descent scales to large sparse primal problems. For one observation, a subgradient with respect to β is

$$\lambda\beta - y_i x_i \mathbb{1}\{y_i(\beta_0 + x_i^\top \beta) < 1\},$$

with a set-valued choice exactly at margin one.

Squared hinge loss is differentiable once but changes the robustness and dual. Kernel methods favor dual solvers, while high-dimensional sparse linear SVMs usually favor primal methods. Optimization tolerance should be judged relative to statistical error: solving the training objective far beyond the precision supported by the data wastes computation.

0.16.3 Compressed Sensing

Compressed sensing recovers a sparse vector $x_0 \in \mathbb{R}^d$ from $b = Ax_0 + \varepsilon$ with fewer measurements than ambient coordinates. The ideal problem

$$\min_x \|x\|_0 \quad \text{subject to} \quad Ax = b$$

is combinatorial. Basis pursuit replaces cardinality by the convex ℓ_1 norm:

$$\min_x \|x\|_1 \quad \text{subject to} \quad Ax = b. \quad (0.16.2)$$

With noise, common forms are

$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1 \quad \text{or} \quad \min_x \|x\|_1 \quad \text{subject to} \quad \|Ax - b\| \leq \delta.$$

ISTA, FISTA, ADMM, and coordinate descent apply directly.

Definition 0.16.1 (Restricted isometry property). A matrix A has restricted isometry constant δ_s if δ_s is the smallest number such that

$$(1 - \delta_s) \|x\|^2 \leq \|Ax\|^2 \leq (1 + \delta_s) \|x\|^2$$

for every s -sparse x .

Theorem 0.16.1 (Sparse recovery, representative form). *If A satisfies a sufficiently strong restricted isometry condition, such as $\delta_{2s} < \sqrt{2} - 1$, then basis pursuit exactly recovers every s -sparse vector from noiseless measurements. With noise and approximate sparsity, the reconstruction error is bounded by a noise term plus the best s -term approximation error.*

Random sub-Gaussian measurement matrices satisfy suitable RIP conditions with roughly $s \log(d/s)$ measurements. Feature standardization and the choice of λ remain important in practical statistical problems.

0.16.4 Deep Learning

A neural network is a composition

$$h_0 = x, \quad z_\ell = W_\ell h_{\ell-1} + b_\ell, \quad h_\ell = \sigma_\ell(z_\ell),$$

trained by minimizing an empirical loss, usually with regularization. Reverse- mode AD is backpropagation applied to this graph.

The objective is nonconvex and has symmetries: hidden units may be permuted, and homogeneous networks allow compensating rescalings across layers. Consequently parameter identifiability and Hessian positive definiteness are not expected. Practical optimization seeks a useful low-loss stationary region, not a certified global minimum.

Initialization controls signal and gradient propagation. Xavier scaling is suited to approximately symmetric activations; He scaling is suited to ReLU-like activations. Normalization, residual connections, and careful parameter scaling improve conditioning. Gradient clipping controls rare large updates but changes the optimization direction.

Minibatch SGD and Adam-type methods dominate because full gradients are expensive. A typical training loop separates:

1. forward evaluation and loss computation;
2. reverse-mode gradient computation;
3. optional clipping or gradient accumulation;
4. optimizer and learning-rate-schedule updates;
5. validation under evaluation-mode behavior.

Weight decay, data augmentation, dropout, and early stopping are regularization choices as well as optimization choices. Training loss alone cannot choose them. Reproducible computation records random seeds, data order, precision, optimizer conventions, schedules, and checkpoint selection.

Mixed-precision training increases throughput but can underflow small gradients; loss scaling and higher-precision accumulators address this. Distributed data-parallel training averages gradients across workers. Very large batches reduce gradient noise but often require learning-rate and warmup adjustments.

Remark 0.16.1. For both classical and deep models, optimization error, estimation error, and approximation error are distinct. A lower training objective does not imply a lower population risk, and an optimizer comparison is meaningful only when data processing, model, tuning budget, and stopping rules are controlled.